Systèmes Concurrents

1h30, documents autorisés

3 décembre 2024

Les réponses doivent être justifiées.

1 Programmation concurrente (4,5 points)

1. Dans le TP sur le problème des philosophes réalisé avec un moniteur, la première approche proposée peut conduit à la famine d'un philosophe, c'est-à-dire un philosophe qui est demandeur et ne parvient jamais à manger. On rappelle qu'un philosophe ne peut pas rester mangeur indéfiniment mais doit reposer les fourchettes après un certain temps (non défini) avant de pouvoir les redemander. Donner un scénario (une exécution) où une telle famine se produit.

   Quatre philosophes A,B,C,D avec C passif, et A entouré par les deux méchants B et D qui s'assurent qu'à tout moment, au moins l'un des deux mange (sans qu'aucun des deux ne mange continûment : c'est illégal). Le philosophe A ne peut alors jamais manger.

2. Dans le TP sur la voie unique réalisé avec des canaux (TP CSP), est-il possible de se retrouver en situation d'interblocage, c'est-à-dire une situation où il y a des demandeurs mais aucun ne parvient à passer, car chaque demandeur attend qu'un autre demandeur passe avant lui ?

   Pas d'interblocage, il y a toujours des trains qui pourront faire une action : si la voie est libre, un train peut entrer ; si la voie est occupée, les trains dans le même sens peuvent entrer, et les trains sur la voie peuvent sortir.

   Autre raisonnement : il n'y a qu'une seule ressource partagée (la voie unique) → il est impossible de construire un cycle dans le graphe d'allocation.

   Il y a par contre risque de famine dans la solution naïve (flot continu de trains dans le même sens, avec toujours au moins un train présent).

3. On considère un ordinateur portable multi-processeurs utilisé par une personne seule qui l'utilise pour rédiger un document multi-langues. En parallèle avec l'éditeur, plusieurs correcteurs orthographiques fonctionnent en arrière-plan, ainsi que des *grands modèles de language* (Large Language Models, de type ChatGPT) pour l'assister dans la rédaction. On considère que l'éditeur est peu consommateur de ressources CPU et de mémoire, que les correcteurs orthographiques sont modérément consommateurs de CPU et peu de mémoire, et que les LLM sont gros consommateurs de CPU et de mémoire. Comment faut-il structurer l'application pour utiliser au mieux les ressources tout en garantissant à l'utilisateur que son éditeur reste réactif ?

   – Préciser en particulier s'il faut un ou plusieurs pools d'activités ?

- Pour chaque pool identifié, quelles caractéristiques a-t-il (fixe ou dynamique ? avec priorité ? si fixe, combien d'activités ? si dynamique, borné ? etc)
- Si on utilise plusieurs pools, est-il intéressant de ségréguer les différents pools, en leur réservant statiquement une partie des ressources du serveur (par exemple certains CPU ou une partie de la mémoire) ?

TODO

2 Problème : gestion du foyer

On considère la gestion du foyer [1] avec des activités « étudiants » et une activité unique « directrice des études ». Les règles d'accès au foyer sont les suivantes :

- Un nombre quelconque d'étudiants peuvent être présents dans le foyer.
- La directrice des études ne peut entrer dans le foyer que s'il n'y aucun étudiant (pour une inspection des locaux) ou s'il y a plus de 50 étudiants (pour arrêter la soirée).
- Tant que la directrice des études est au foyer, aucun étudiant ne peut y entrer, mais les étudiants présents peuvent en sortir.
- La directrice des études ne peut pas sortir du foyer tant qu'il reste des étudiants.

2.1 Synchronisation par moniteur, sans stratégie (9 points)

Développer un moniteur qui réalise la gestion de ce foyer en respectant la méthodologie :

1. Déterminer l'interface du moniteur ;
2. Énoncer informellement les prédicats d'acceptation de chaque opération ;
3. Déduire les variables d'état qui permettent d'écrire ces prédicats d'acceptation ;
4. Formuler l'invariant du moniteur et les prédicats d'acceptation ;
5. Associer à chaque prédicat d'acceptation une variable condition qui permettra d'attendre/signaler la validité du prédicat ;
6. Programmer les opérations, en suivant le schéma présenté en cours ; 3 pt
7. Vérifier que le moniteur est correct (= montrer que les appels à `signal` sont corrects).

1. *Interface : EntrerEtu, SortirEtu, EntrerDir, SortirDir*
2. *Prédicats :*
   - *EntrerEtu : directrice pas présente*
   - *SortirEtu : –*
   - *EntrerDir : $nbEtu = 0 \vee nbEtu > 50$*
   - *SortirDir : $nbEtu = 0$*
3. *Variables d'état : nbEtu, dirPrésente*
4. *invariant : rien ! (typage : $nbEtu \in Nat$, $dirPrésente \in Bool$)*
5. *Variables conditions : entrerE, entrerD, sortirD*

1. L'examen a été rédigé avant la fermeture du foyer, c'est une coïncidence.

6.

```
EntrerEtu : si dirPrésente alors entrerE.wait; finsi
            nbEtu++;
            entrerE.signal    // réveil en chaîne
            si nbEtu > 50 alors entrerD.signal; finsi
SortirEtu : nbEtu--
            si nbEtu = 0 alors
                si dirPrésente alors sortirD.signal;
                               sinon entrerD.signal; finsi
            finsi
EntrerDir : si nbEtu > 0 && nbEtu < 50 alors entrerD.wait; finsi
            dirPrésente := true
SortirDir : si nbEtu > 0 alors sortirD.wait; finsi
            dirPrésente := false;
            entrerE.signal
```

7. Vérification : il aurait fallu écrire les pré- et post-conditions, les implications sont toutes triviales. J'ai eu la flemme, je perds un point.

Note 1 : comme l'activité "directrice des études" est unique, on peut écrire :

```
SortirEtu : nbEtu--
            si nbEtu = 0 alors
                sortirD.signal;
                entrerD.signal;
            finsi
```

Note 2 : sur cet exercice, et parce que l'activité "directrice des études" est unique, on pourrait fusionner `entrerD` et `sortirD` en une seule variable condition `bougerD`. Cependant, ce n'est pas la méthodologie (les conditions d'acceptation pour EntrerDir et SortirDir ne sont pas les mêmes) et en outre, la vérification échouera : `bougerD.signal` dans EntrerEtu aurait pour pré-condition `not dirPrésente and nbEtu > 50`, alors que la post-condition dans SortirDir est `dirPrésente and nbEtu = 0`. Mais cette situation (passage du seuil de 50, donc entrée d'un nouvel étudiant, alors que la directrice attend de sortir) est impossible.

2.2 Processus communicants (6,5 points)

L'objectif est de résoudre le problème en utilisant des canaux et une activité de synchronisation.

Questions Répondre en utilisant du langage algorithmique, du pseudo-Go ou la syntaxe Java-CSP du TP. La syntaxe précise est sans importance du moment que l'intention est claire.

1. Identifier les canaux nécessaires.. 1 pt
2. Donner le code de la directrice des études. 1 pt
3. Donner l'activité de synchronisation, en utilisant au choix l'approche condition ou l'approche automate (conseil : l'approche condition est moins verbeuse). 4,5 pt

1. entrerE, sortirE, entrerD, sortirD

2. *boucle entrerD.send(_) ; sortirD.send(_) ; finboucle*
3. *En pseudo-code :*

```
var nbEtu, dirPrésente := 0, false;
loop
    select
        when ¬dirPrésente => entrerE.receive(); nbEtu++;
    or
        sortirE.receive(); nbEtu--;
    or
        when nbEtu = 0 or nbEtu > 50 => entrerD.receive(); dirPrésente := true;
    or
        when nbEtu = 0 => sortirD.receive(); dirPrésente := false;
    end select
end loop
```

En Go :

```
func synchroFoyer() {
  nbEtu := 0;
  dirPrésente := false;
  for {
    select {
        case <- when(!dirPrésente, entrerE) : nbEtu++;
        case <- sortirE : nbEtu--;
        case <- when(nbEtu = 0 || nbEtu > 50, entrerD) : dirPrésente = true;
        case <- when(nbEtu = 0, sortirD) : dirPrésente = false;
    }
  }
}
```

Avec un automate strict, il faut cinq états : personne (entrerE, entrerD), moins de 50 étudiants (entrerE, sortirE), plus de 50 étudiants (entrerE, sortirE, entrerD), directrice présente avec des étudiants (sortirE), directrice présente sans étudiant (sortirD).
En mixant avec des conditions, des automates à trois ou quatre états peuvent aussi être corrects.